SDP Project — Notes Part 2 (Branching Strategy & Daily Git Commands)

Continuation of `sdp-project-notes.md`. Covers what was learned/fixed after the first PR merge — mainly around daily workflow, `dev` branch setup, and commands that are easy to forget.

---

1. Problem: Forgetting the push workflow

What happened: After merging the first PR, a new commit (adding docs) was made directly on `main`. Pushing it directly failed again (protected branch), and doing a full branch + PR cycle for every small change felt like too much overhead for solo work.

Realization: Branch protection was only ever meant to apply to `main`. The fix isn't to avoid protection — it's to **stop working directly on `main` at all** and do daily work on an unprotected branch instead.

---

2. Problem: `dev` branch didn't exist

What happened: Running `git branch` showed only `main` and `fix/flatten-repo-structure` — no `dev`. This was because the repo had been set up from a plain zip (without git history) and re-initialized fresh, so `dev` (which was only mentioned in planning/docs) was never actually created as a real branch.

Fix:

```
git checkout -b dev       # create dev branch from current state of main
git push origin dev       # push it to GitHub so it exists remotely too
```

Key learning: A branch mentioned in documentation isn't real until it's actually created with `git checkout -b` (or `git branch`) and pushed. Docs describe the *intended* workflow — they don't create the branches themselves.

---

3. Concept: Why a new branch has all the same files as `main`

Question raised: Why does `dev` already contain everything that was in `main`?

Answer: A new branch is created as a **snapshot/pointer at the current commit** — not an empty folder. `git checkout -b dev` means *"create a new branch starting from exactly where I am right now."* All existing files/history come with it automatically. After that point, `main` and `dev` can diverge independently as each gets new commits.

```
main:  [A] --- [B]
                \
dev:              (starts here, identical to B) --- [C] --- [D]  (new work goes here)
```

This is *why* branching is useful — you get a safe copy to build on top of, instead of starting from scratch.

---

4. Finalized Branching Strategy

```
main   → stable, protected, demo-ready code only
  └── dev   → default working branch for day-to-day commits (NOT protected)
       └── feature/<name>  → only for large/risky changes (new service, big refactor)
```

Rule of thumb:

- Small changes, docs, fixes, day-to-day commits → work directly on `dev`, push directly (no PR needed).
- Large/risky features (e.g. a whole new microservice) → create a `feature/<name>` branch off `dev`, then PR it into `dev`.
- `main` is only updated occasionally, via PR from `dev`, when there's a stable milestone (e.g. "ingestion service complete", "MVP working").

Interview line: *"I used a three-tier branching strategy — `main` for stable/ demo-ready code, `dev` as the daily working branch, and short-lived feature branches for larger changes — so that protection rules only get in the way when they should, not for every small commit."*

---

5. Daily Git Commands Cheat Sheet

Basic push flow (most common — used on `dev`)

```
git checkout dev                          # switch to dev branch
git pull origin dev                       # get latest changes first
git add .                                 # stage all changed files
git commit -m "type: short description" # commit with a message
git push origin dev                       # push directly (dev isn't protected)
```

Creating a new branch

```
git checkout -b <branch-name>           # create + switch to new branch in one step
git push origin <branch-name>           # push it to GitHub (first time)
```

Checking status

```
git branch                              # list branches, * shows current one
git status                              # see staged/unstaged changes
git log --oneline                       # compact commit history
```

Syncing after a PR merge (for `main`)

```
git checkout main
git pull origin main                    # bring merged changes into local main
```

Deleting a branch once its work is merged (cleanup)

```
git branch -d <branch-name>                 # delete locally
git push origin --delete <branch-name>      # delete on GitHub too
```

Moving files (used to fix the nested folder issue)

```
git mv <source> <destination>           # move/rename a tracked file or folder
```

---

6. Commit Message Convention (Conventional Commits)

Prefix	Use case
`feat:`	New feature
`fix:`	Bug fix
`docs:`	Documentation only changes
`chore:`	Maintenance (config, structure, cleanup)
`refactor:`	Code change that isn't a fix or feature
`test:`	Adding/updating tests

Example: `git commit -m "docs: add setup notes for branching strategy"`

---

7. Key Pointers to Remember

- **Never work directly on `main`.** Always be on `dev` (or a feature branch) for daily commits.
- **`main` is protected on purpose** — it's not a bug when a direct push gets rejected, it's the safety net working.
- **A new branch = same content as the branch it was created from**, not empty — branches diverge only after new commits.
- **PRs matter even solo** because they trigger CI checks and create a clear history — but "Require approvals" should be OFF for solo projects since there's no second reviewer.
- **`git checkout -b`** creates *and* switches to a branch in one command — saves a step vs `git branch` + `git checkout`.
- When in doubt, run `git branch` first to confirm which branch you're actually on before adding/committing/pushing.